



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

FPGA ACCELERATOR FOR LLM

REPORT OF WEEK 2

Work of :

GAUVRIT Jean-Luc

As intern student

With the supervision of :

KIM Tae-Wan

JUNE 2025

Contents

1	Introduction	2
1.1	Presentation of the Weekly Report	2
1.2	Presentation of the Subject	2
1.3	The model	2
1.4	Goals	2
1.5	FPGA	3
2	Milestones	4
3	Research and Preparation	5
3.1	Model	5
3.2	FPGA evaluation kit	5
4	First inference	7
4.1	Preparation	7
4.1.1	Convert PyTorch files	7
5	Optimization	8
5.1	Softmax	8
5.1.1	Temperature	8
5.1.2	Taylor's Theorem	9
5.1.3	Approximation of Softmax without Division	10
5.2	Normalization Algorithms for Inference Stability	10
5.2.1	Standard Softmax	10
5.2.2	Fast Softmax using Taylor Expansion	10
5.2.3	FlashNorm (RMS Normalization)	11
5.2.4	LogSumExp Function	11
6	Conclusion	12
	Bibliography	13

List of Figures

1	Logo de DeepSeek	2
2	Milestones for this project	4
3	Argument for the choice of FPGA	5
4	Visualizing the Effects of Temperature Scaling	9

1 Introduction

1.1 Presentation of the Weekly Report

This weekly report presents the progress of my work on designing and optimizing a hardware accelerator for Large Language Models (LLMs) using Field-Programmable Gate Arrays (FPGAs). Its objective is to document technical advancements, highlight open design questions, and ensure alignment with the research direction defined in collaboration with Professor Tae-Wan Kim. Each section outlines key investigations, experimental results, and technical challenges encountered during the week.

1.2 Presentation of the Subject

The goal of this project is to do inference with a LLM, specifically a variant of the DeepSeek-R1 family, on a FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.

LLMs typically require considerable computational power and memory bandwidth, which limits their deployment in edge or low-power environments. While GPUs provide strong performance, their energy consumption is often prohibitive for embedded or mobile use cases.

This project explores the use of FPGAs to design a custom, energy-efficient accelerator optimized for DeepSeek inference. The flexibility of FPGAs allows for fine-tuned architectural choices targeting both compute and memory efficiency. Our focus is on reducing power consumption while maintaining acceptable latency (e.g., 100–200 ms per token), with attention given to constraints such as voltage levels, memory interfaces, and model size.

1.3 The model

The goal of this project is to perform inference with a Large Language Model, specifically a variant of the DeepSeek-R1 family, on an FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.



Figure 1: Logo de DeepSeek

Ultimately, the objective is to evaluate whether FPGA-based acceleration can provide a sustainable and scalable solution for running LLMs like DeepSeek in compact or embedded systems.

1.4 Goals

Our final goal is to develop an Application-Specific Integrated Circuit (ASIC) that is energy-efficient and capable of accelerating LLMs in low-power environments. The development of a

Proof of Concept (PoC) using FPGAs is a crucial step towards achieving this goal. The PoC will demonstrate the feasibility and potential benefits of using FPGAs for LLM acceleration, paving the way for future ASIC development.

1.5 FPGA

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits that can be programmed to implement custom digital logic. Their main advantage lies in flexibility : unlike fixed-function hardware, FPGAs allow designers to create architectures tailored to specific applications, such as neural network inference. This adaptability makes them well suited for achieving an efficient balance between performance, power consumption, and resource usage.

2 Milestones

The milestones presented here offer a broad overview of the project's key phases.



Figure 2: Milestones for this project

In fact, each milestone encompasses numerous intermediate steps and detailed tasks that are essential for the successful completion of the project. The process is iterative, particularly in the optimization and Proof of Concept phases, where continuous refinement and testing are crucial.

3 Research and Preparation

3.1 Model

Our focus is on DeepSeek due to its superior performance in terms of efficiency relative to the number of parameters compared to other LLMs. This makes DeepSeek an excellent candidate for FPGA-based inference.

DeepSeek-R1 was released with several model variants, including distilled versions that reduce the number of parameters while maintaining performance. Although these distilled variants are interesting, they are not suitable for our purposes because we intend to use FPGA techniques that compress the transformer.

The aim is for the final Proof of Concept (PoC) to be capable of running the original version of DeepSeek-R1. **However, to optimize the development phase up until the measuring phase, I plan to use the DeepSeek-Coder-1.3B-Base model.**

3.2 FPGA evaluation kit

I found two FPGA evaluation kit who we can use for this project. We must compare two FPGA solutions from AMD: the Virtex UltraScale+ VCU118 and the Zynq UltraScale+ MPSoC ZCU102, focusing on their energy efficiency and performance for applications involving LLMs. The goal is to maintain an inference time of at least 100ms per token while efficiently managing memory.

The Virtex UltraScale+ VCU118 kit is designed for high-performance applications, featuring a large FPGA fabric, high-speed transceivers, and extensive memory interfaces, making it suitable for data centers and high-performance computing. In contrast, the Zynq UltraScale+ MPSoC ZCU102 kit combines a processing system with programmable logic, offering a balance of performance and power efficiency, ideal for embedded applications like machine learning.

In terms of energy efficiency, the Virtex kit is optimized for high throughput but can have higher power consumption, whereas the Zynq kit, with its integrated design, allows for more fine-grained power management, leading to reduced power consumption.

For performance, the Virtex kit offers higher computational resources and memory bandwidth, making it suitable for applications requiring high parallelism. The Zynq kit, however, provides efficient task offloading between CPU and FPGA, optimizing inference times but may face limitations with very large models due to its smaller FPGA fabric.

Efficient memory management is crucial for both performance and power consumption. The Virtex kit, with its high-speed memory interfaces and large on-chip memory, is beneficial for large-scale applications. The Zynq kit features tightly coupled memory interfaces between the processing system and programmable logic, optimizing data transfer and reducing latency.

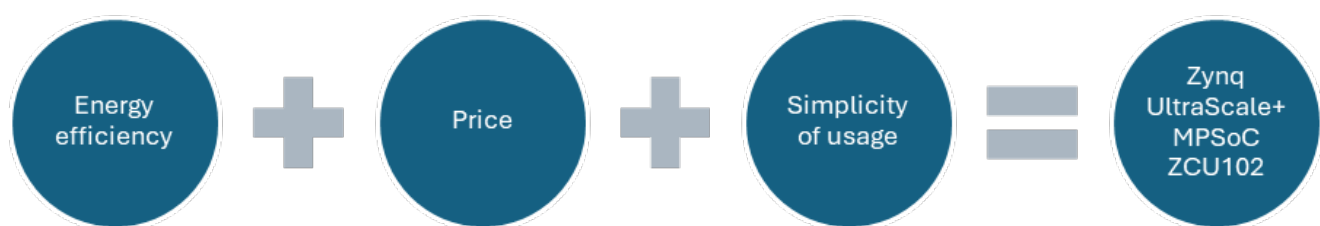


Figure 3: Argument for the choice of FPGA

In conclusion, the choice between the two kits depends on specific application requirements. The Virtex UltraScale+ VCU118 is ideal for maximizing performance, while the Zynq UltraScale+ MPSoC ZCU102 is better suited for applications prioritizing energy efficiency and integration with a processing system. Both kits offer robust memory management features to meet the 100ms/token inference requirement.

4 First inference

4.1 Preparation

Throughout this process, we utilize the DeepSeek-Coder-1.3B-Base model. This initial phase is crucial for setting the foundation for future work with larger models. We meticulously save all command lines and Python code used during this stage. This documentation is essential as it prepares us for the inference of more complex and larger models, ensuring that we have a clear and reproducible workflow.

4.1.1 Convert PyTorch files

A primary challenge in deploying transformer models on FPGA is their incompatibility with standard machine learning frameworks, such as **PyTorch**, and the associated tensor weight formats (e.g., `.safetensors` files). FPGA require models to be in a hardware-agnostic format that supports optimization for specialized accelerators. To address this, the model is converted from its native PyTorch format to the ONNX¹ format, an open standard that enables interoperability across diverse frameworks and hardware platforms.

The conversion process involves several key steps, implemented in Python using the HuggingFace transformers library and PyTorch. First, the pre-trained model and its corresponding tokenizer are loaded from the HuggingFace Hub. A dummy input, tokenized from the string “Hello, world!”, is prepared to trace the model’s computational graph during export. To ensure compatibility with ONNX, a wrapper class is defined to adapt the model’s forward method, disabling the use of caching mechanisms (e.g., key-value caches for autoregressive generation) that are not supported in standard ONNX exports. The model is then exported to ONNX format using PyTorch’s `torch.onnx.export` function, with specific configurations to support dynamic input shapes and compatibility with FPGA inference.

The following Python script illustrates the complete conversion process:

```
1 torch.onnx.export(  
2     wrapped_model,  
3     (dummy_input["input_ids"],),  
4     "deepseek.onnx",  
5     input_names=["input_ids"],  
6     output_names=["logits"],  
7     opset_version=16, # Use opset 16 for compatibility  
8     dynamic_axes={  
9         "input_ids":  
10             {0: "batch_size", 1: "sequence"},  
11         "logits": {0: "batch_size", 1: "sequence"}  
12     }  
13 )
```

ONNX provides an open standard for representing machine learning models, making it easier to transition between different frameworks and hardware.

¹The Open Neural Network Exchange (ONNX) is an open-source artificial intelligence ecosystem of technology companies and research organizations that establish open standards for representing machine learning algorithms and software tools to promote innovation and collaboration in the AI sector.

5 Optimization

5.1 Softmax

Softmax is a mathematical function that normalizes input scores into a probability distribution, making the outputs sum to 1. In attention mechanisms, it is used to compute attention weights, highlighting the relative importance of different elements in a sequence.

Mathematically, it is defined as follows :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (1)$$

Here, x_i denotes the input scores. An important property of the softmax function is its invariance to additive constants :

$$\text{softmax}(Ax + B) = \text{softmax}(Ax)$$

This allows us to subtract the maximum value of the input vector without changing the result. Defining $x' = x - \max(x)$ ensures numerical stability, as each component of $e^{x'}$ then lies in the interval $0 < e^{x'} \leq 1$.

$$\begin{aligned} \text{softmax}(x') &= \text{softmax}(x - \max(x)) \\ &= \frac{\exp(x_i - \max(x))}{\sum_j^n \exp(x_j - \max(x))} \text{ with } e^{a+b} = e^a e^b \\ &= \frac{e^{-\max(x)}}{e^{-\max(x)}} \times \frac{e^{x_i}}{\sum_j e^{x_j}} \\ &= \text{softmax}(x) \end{aligned}$$

This technique, often called **log-sum-exp trick**, improves numerical stability by preventing large exponentials, ensuring that $0 < e^{x_i - \max(x)} \leq 1$.

5.1.1 Temperature

The generated sequence often exhibits a predictable and uniform structure due to reduced entropy in the softmax distribution. Specifically, the probability of selecting a particular word. While predictability is not inherently problematic when the objective is to produce realistic sequences, it becomes a limitation when the goal is to generate new text or images that are unique and previously unseen. In such cases, introducing randomness is essential to achieve creative outputs.

To address this, increasing the randomness of the sampling process is necessary. Temperature scaling provides a mechanism to control the entropy of the probability distribution used for sampling, thereby influencing the predictability of the next word. This is achieved by scaling the logit vector by a temperature parameter, denoted as temperature. This parameter T modulates the distribution's sharpness, allowing for a balance between predictability and novelty in the generated sequences.

$$\text{Softmax}(x_i) = \frac{e^{\frac{x_i}{T}}}{\sum_j e^{\frac{x_j}{T}}}$$

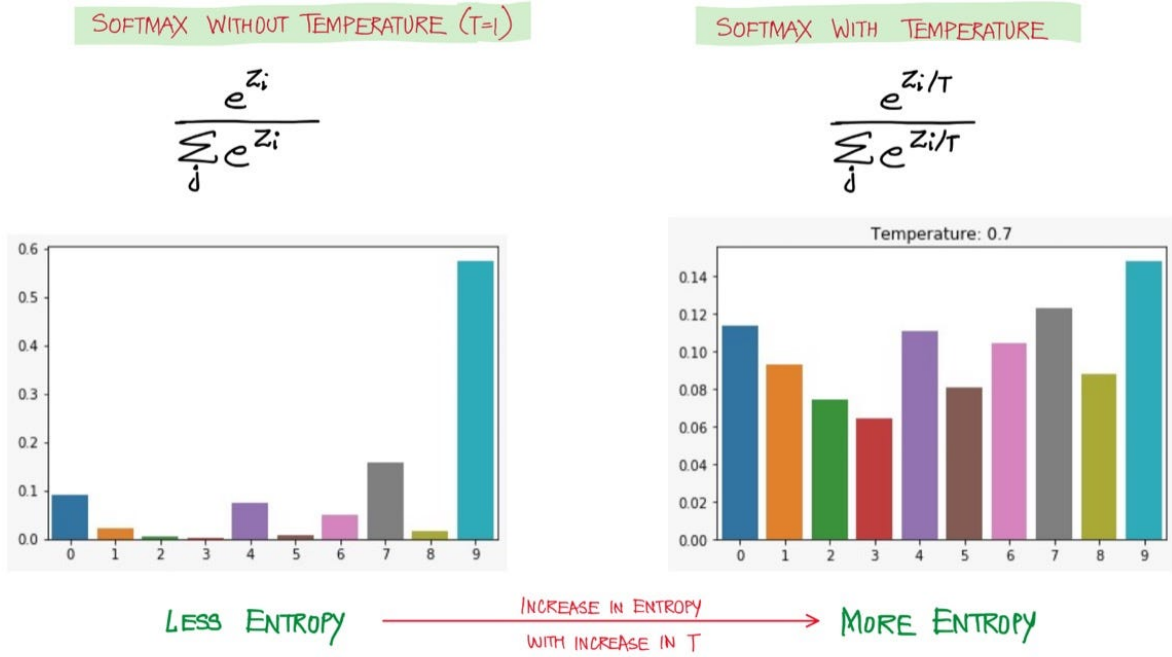


Figure 4: Visualizing the Effects of Temperature Scaling

This results 4 in greater diversity in the generated sequence, enhancing its novelty. However, excessive diversity can lead to undesirable outcomes, such as sequences that lack coherence or meaning. Therefore, like other hyperparameters, the temperature parameter must be carefully tuned to balance novelty and coherence.

Typically, the temperature parameter T is set as a constant value in the design of LLM. To streamline the hardware implementation and avoid the computational overhead associated with division operations, **the LLM architecture is designed with a fixed temperature of $T = 1$** . This simplification reduces hardware complexity while maintaining the desired performance characteristics of the model.

5.1.2 Taylor's Theorem

In calculus, Taylor's theorem provides a way to approximate a function that is k -times differentiable near a point a using a polynomial of degree k , known as the k -th order Taylor polynomial. For sufficiently smooth functions, this polynomial corresponds to the truncation of the Taylor series at order k :

$$\begin{aligned} f(x) &= f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f^{(2)}(a)}{2!}(x-a)^2 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \sigma_n(x) \\ &= \left(\sum_{k=0}^n \frac{f^{(k)}(a)}{k!}(x-a)^k \right) + \sigma_n(x) \end{aligned}$$

In our context, we aim to optimize the **softmax** function, which relies heavily on the exponential function e^x . For small values of x (i.e., as $x \rightarrow 0^+$) and with a truncation order of $k = 2$, the Taylor expansion of e^x around 0 yields:

$$e^x \approx \sum_{k=0}^2 \frac{x^k}{k!} = 1 + x + \frac{x^2}{2}$$

This approximation allows us to reduce computational complexity by replacing the exponential with a second-order polynomial, which is significantly more efficient to evaluate on hardware such as FPGAs.

However, note that the `softmax` function also involves a division step (normalization), which remains computationally expensive. While the Taylor expansion helps reduce the cost of computing e^x , **a more comprehensive optimization is still required to eliminate or simplify the division operation in order to fully optimize softmax for hardware deployment.**

5.1.3 Approximation of Softmax without Division

[1]

5.2 Normalization Algorithms for Inference Stability

In this section, we present and analyze several normalization algorithms used to improve numerical stability and computational efficiency in large-scale inference tasks, particularly in the context of softmax-based attention mechanisms.

5.2.1 Standard Softmax

The standard softmax function transforms input scores into a probability distribution by applying the exponential function followed by normalization:

```

1 def softmax(x):
2     """Compute the softmax of a tensor."""
3     e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
4     return e_x / np.sum(e_x, axis=-1, keepdims=True)

```

To improve numerical stability and avoid overflow in the exponential calculation, we subtract the maximum value of the input. This technique, often referred to as the *log-sum-exp trick*, ensures that the exponentials are bounded between 0 and 1, while preserving the output distribution.

5.2.2 Fast Softmax using Taylor Expansion

To reduce computational cost, we propose approximating e^x using a truncated Taylor series:

$$e^x \approx 1 + x + \frac{x^2}{2}$$

This leads to the following softmax approximation:

```

1 def fast_softmax_taylor(x):
2     """
3     Approximate softmax using Taylor expansion of e^x:
4     e^x ≈ 1 + x + x^2/2
5     """
6     approx = 1 + x + 0.5 * x**2
7     return approx / np.sum(approx, axis=-1, keepdims=True)

```

This method trades accuracy for performance and is particularly useful in resource-constrained environments such as FPGA inference.

5.2.3 FlashNorm (RMS Normalization)

FlashNorm is an alternative normalization technique based on the Root Mean Square (RMS) of the input. It simplifies traditional layer normalization while preserving the essential scaling properties:

```
1 def FlashNorm(x, eps=1e-6):
2     """
3     FlashNorm: simplified RMS normalization
4     x: input tensor (... , d)
5     Returns: normalized tensor of same shape
6     """
7     rss = np.sqrt(np.sum(x ** 2, axis=-1, keepdims=True) + eps)
8     gain = np.ones_like(x)
9     return (x / rss) * gain
```

This technique avoids computing mean and variance separately, making it more efficient for hardware deployment.

5.2.4 LogSumExp Function

The LogSumExp function is often used in log-domain computations to maintain numerical stability:

```
1 def LogSumExp(x):
2     """
3     Compute the LogSumExp of a tensor.
4     """
5     xmax = np.max(x, axis=-1, keepdims=True)
6     return xmax + np.log(np.sum(np.exp(x - xmax), axis=-1, keepdims=True))
```

It is particularly useful for computing log-likelihoods and normalizing log probabilities.

6 Conclusion

This second week of the internship has laid a strong foundation for the development of an FPGA-based accelerator for Large Language Models. Significant progress has been made in understanding the DeepSeek architecture, selecting suitable FPGA evaluation kits, and exploring optimization strategies for softmax and normalization functions. In particular, mathematical approximations such as the Taylor expansion and FlashNorm have been implemented and tested to assess their hardware feasibility.

The transition from PyTorch to ONNX format has also been successfully initiated, preparing the ground for future synthesis on FPGA platforms. Additionally, a careful comparison between high-end (VCU118) and edge-oriented (ZCU102) FPGA kits was carried out to align hardware selection with our performance and energy goals.

The next step will involve validating the inference on simplified models, then optimizing the memory and computational pipeline in HLS/Vivado. The ultimate objective remains to produce a proof-of-concept accelerator capable of demonstrating real-time inference performance with minimal power consumption.

This ongoing work reflects a balance between algorithmic innovation and hardware practicality—key to enabling efficient deployment of LLMs in embedded systems.

References

- [1] Nils Graef, Andrew Wasielewski, and Matthew Clapp. Flashnorm: fast normalization for llms, 2025.